

5-11

hash limitations

- Se Eve intercetta la comunicazione che contiene testo e digest (M, D) ,
- può modificare M in M' e ricalcolare il digest $H(M') = D'$.

necessità di due canali

Usare un hash per garantire l'integrità ha senso solo se il digest è inviato in un canale diverso da quello usato per il messaggio. L'hash è efficace se l'attaccante non può intercettare entrambi i canali.

HMAC (Hash-based Message Authentication Code)

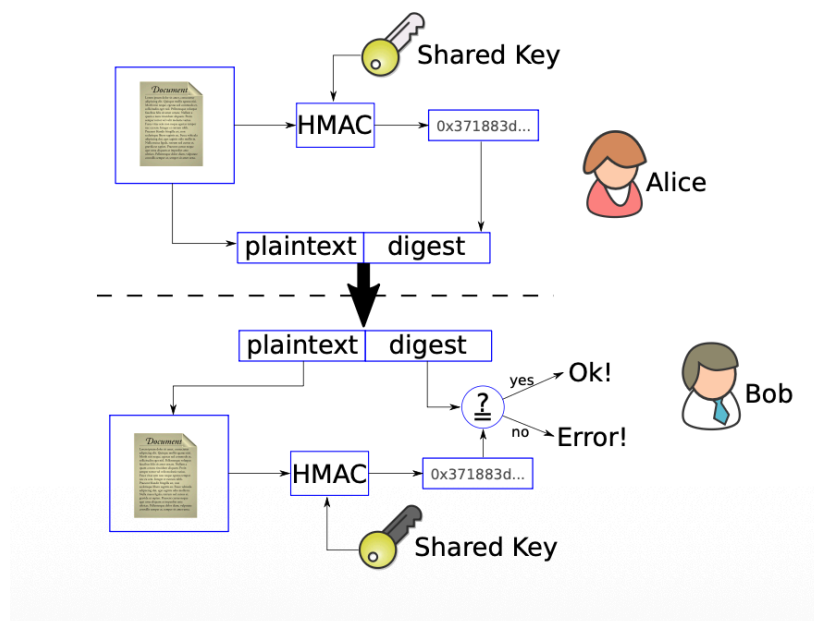
Per prima cosa Alice e Bob concordano una chiave segreta K e una funzione di hash $H()$.

Possono usare la password come input aggiuntivo alla funzione di hash.

ipad = inner padding, opad = outer padding

$$D = \text{HMAC}(M, K) = H((K \oplus \text{opad}) \mid H((K \oplus \text{ipad}) \mid M))$$

- opad e ipad sono sequenze necessarie per allineare K alla lunghezza dell'output di $H()$. Se K è più lunga del digest, allora K è rimpiazzata con $H(K)$.



Anche se Eve intercetta sia M sia D , per rompere l'integrità del messaggio dovrebbe modificare M in M' ma, per ricalcolare HMAC , avrebbe bisogno di conoscere K .

pro

$HMAC$ può essere inviato nello stesso canale del messaggio e garantisce autenticazione, perché Bob sa che l'unica a conoscere la chiave segreta è Alice e quindi solo lei può generare un $HMAC$ corretto.

cons

Alice e Bob necessitano di un canale sicuro separato per scambiarsi la chiave segreta. Se Eve intercetta quella comunicazione, sarebbero compromesse anche le successive.

generazione della chiave

La chiave non deve essere indovinabile dall'attaccante e deve essere una stringa binaria casuale. Un modo per generare una chiave è attraverso una funzione di hash:

- I due enti concordano una password condivisa P , poi generano $K = H(P)$.



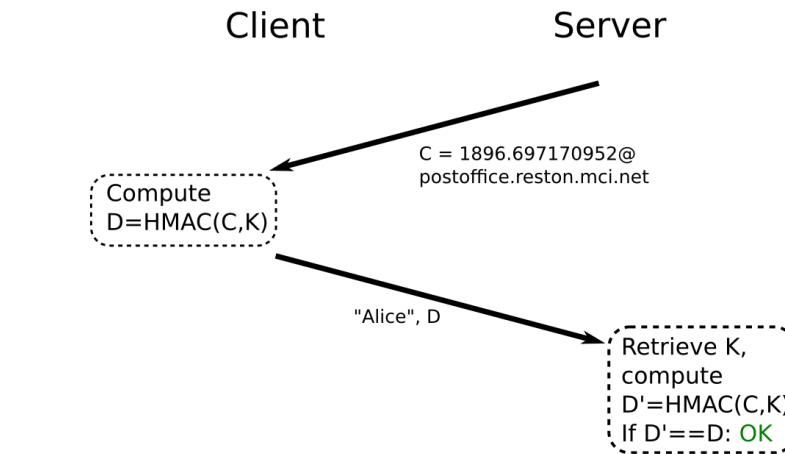
Eve può comunque fare brute force dell'hash, quindi la password deve essere sicura.

- La password deve avere almeno 15–20 caratteri e includere lettere, numeri e simboli. Aumentare l'alfabeto rende più difficile enumerare tutte le password.
- Non deve comparire in un dizionario di parole esistenti.

CRAM-MD5

È un protocollo che ottiene l'autenticazione dell'utente, basato sulla conoscenza di una password e di una funzione di hash condivise.

- Alice vuole autenticarsi a Bob. L'username è "Alice" e la password condivisa è K .
- Bob invia una challenge, una stringa casuale, spesso un timestamp.
- Alice, che conosce K , computa $D = HMAC(C, K)$ e invia a Bob il messaggio ("Alice", D). Bob controlla se "Alice" è presente nel database, recupera K , ricomputa $D' = HMAC(C, K)$ e verifica che $D == D'$.

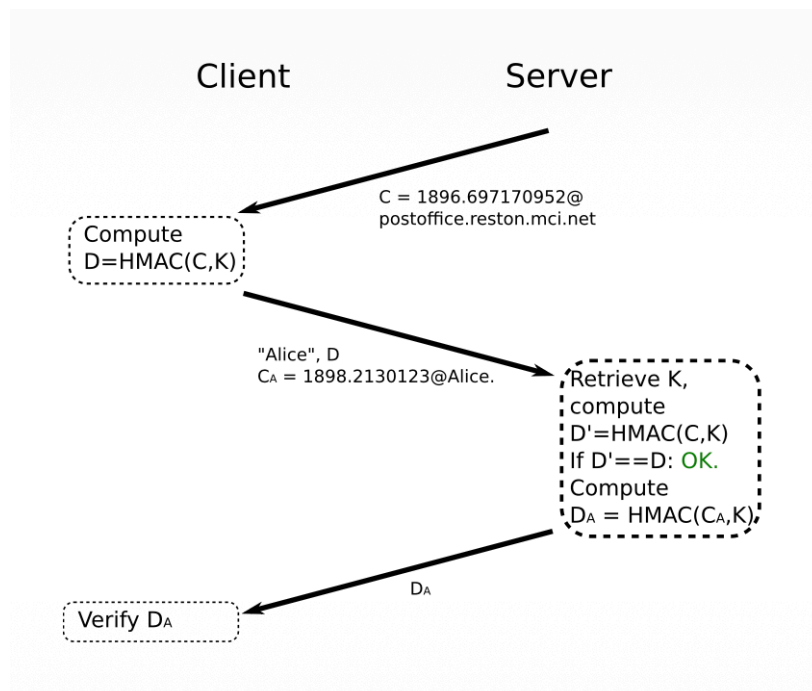


È importante che la challenge sia sempre diversa, in modo che Eve non possa fare un replay attack riutilizzando D intercettato in uno scambio precedente.



È migliore della trasmissione della chiave segreta in plaintext, ma ora è deprecato per quattro motivi:

1. L'autenticazione non è mutuale:
 - a. Alice non ha garanzia che il server sia Bob.
 - b. Si può ripetere il protocollo invertendo i ruoli per dare ad Alice evidenza che anche Bob conosce la password.
2. *MD5* non è considerato sicuro. Inoltre non è saggio dipendere da una singola funzione di encryption. Se si negoziano più funzioni, si introducono messaggi extra che possono essere manipolati da Eve per forzare l'uso dell'opzione più debole.



1. Attacchi brute force:

- a. Eve può osservare C e D in chiaro, quindi può eseguire un brute-force offline con una lista di password L .



Peggio offline perché non può essere interrotto se identificato.

fondamenti di crittografia

Secondo il principio di **Kerckhoffs**, l'algoritmo di cifratura deve essere pubblico. Il **segreto** risiede nella **chiave**. Non esistono algoritmi "segreti" sicuri a lungo termine; è più sicuro un algoritmo ampiamente analizzato.

sostituzione semplice

L'algoritmo di Cesare è un algoritmo di semplice sostituzione: sostituisce un simbolo con un altro in modo consistente (A è sempre mappato in N). Consente solo 25 shift, quindi il brute force è semplice.



Indipendentemente dal mapping tra cleartext e ciphertext, questa encryption può essere sconfitta con analisi statistica.

Eve prende un insieme di testi in italiano, conta le lettere più frequenti, poi fa lo stesso con il testo cifrato, sostituisce le lettere più frequenti e verifica se l'output è comprensibile.

algoritmo di trasposizione

Nell'algoritmo di trasposizione le colonne sono scambiate secondo una chiave K . La trasposizione preserva la frequenza delle lettere, quindi è semplice inferire la lingua di origine; almeno la stessa lettera X non è sempre mappata alla stessa lettera cifrata.

Si possono anagrammare sequenze di testo per recuperare il pattern di trasposizione.



Se c'è una correlazione statistica tra M e C , Eve potrebbe trovare un modo per ottenere M senza conoscere K .

One-Time Pad (OTP)

Immaginiamo K come chiave binaria lunga almeno quanto il messaggio. $C = M \oplus K$. In questo modo non è possibile risalire a M senza conoscere K .

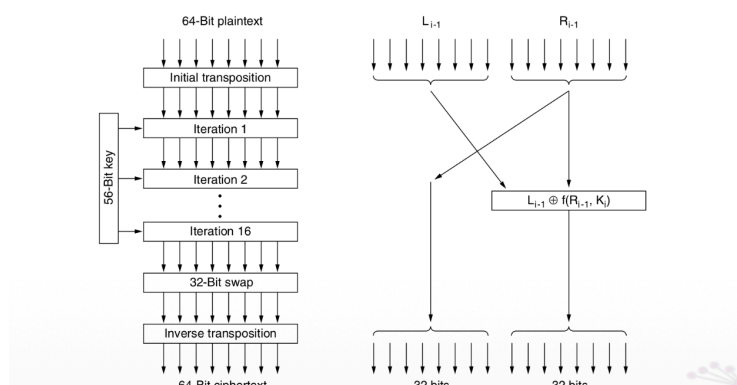
Se Eve ottiene C e prova a fare brute force per trovare la chiave, otterrà tutte le permutazioni di bit del messaggio. Ogni chiave candidata mappa a un solo messaggio, quindi nessun plaintext ha probabilità maggiore degli altri.

👉 La più grande limitazione dell'OTP è che la chiave deve essere lunga quanto M . Se Alice ha un canale sicuro per la chiave, perché non inviare direttamente il messaggio?

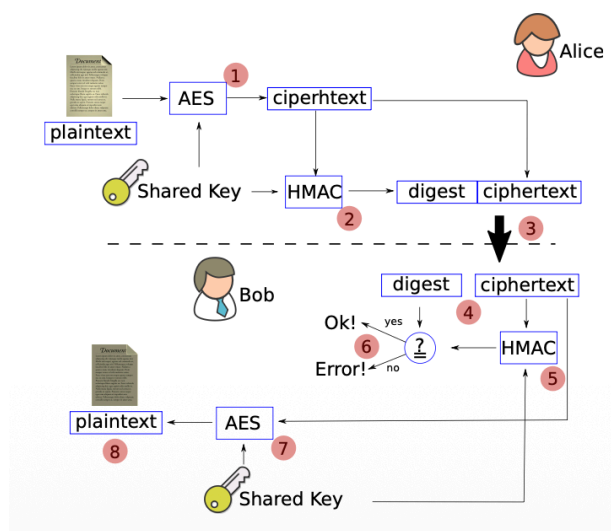
⚠️ Riducendo la lunghezza della chiave in un *OTP* si reintroduce correlazione tra M e C .

algoritmo di Feistel

I cifrari moderni combinano sostituzione e trasposizione per ridurre al minimo la correlazione statistica tra M e C , in modo che risulti computazionalmente impossibile risalire a M .



protocollo sicuro, segreto e con autenticazione



sender

1. Alice usa un algoritmo a chiave simmetrica per trasformare il plaintext in ciphertext, con una chiave condivisa.
2. Il ciphertext è passato a *HMAC*, che riusa la chiave condivisa. Il risultato è un digest del ciphertext.

3. Digest e ciphertext sono concatenati e inviati.

receiver

1. Bob riceve il messaggio e usa il ciphertext con la chiave segreta per verificare il digest.
2. Se il digest corrisponde a quello ricevuto, il messaggio non è stato modificato durante la trasmissione: sono garantite integrità e autenticazione.
3. Bob usa *AES* per decrittare il ciphertext ottenendo il plaintext; la trasmissione è confidenziale perché solo lui e Alice conoscono la chiave.



Il brute force offline rimane un rischio. Usare chiavi separate per *HMAC* ed encryption riduce la superficie d'attacco.

password files

- Una password deve essere memorizzata in un formato che consenta il confronto con l'input dell'utente.
- Non può essere salvata in plaintext: deve essere trasformata in modo tale che, se recuperata da un attaccante, non permetta di risalire alla password originale.

L'hash è adatto per memorizzare le password:

- Gli hash sono ottimi per verificare l'uguaglianza degli input.
- Non sono invertibili.



password leakage

Se Eve viola il database ottiene solo D . Per autenticarsi serve P e, dato che le funzioni di hash non sono reversibili, non può ricavare P da D .

pre-hashed passwords

Se Eve attacca il server e ruba il database delle password (hashate), poi prende un dizionario di m parole e per ciascuna parola p calcola l'hash e lo salva. Alla fine solo $n \ll m$ persone avranno una password debole. Più tardi Eve potrà riutilizzare gli m hash calcolati (la parte computazionalmente onerosa) per confrontarli con un altro database Y di password hashate.

salted hashes

Il salt è un numero casuale aggiunto all'input. Due servizi X e Y avranno salt differenti: in questo modo Eve non può riusare la tabella generata contro X per violare Y .

slow hashes

Per aumentare il costo computazionale dell'hashing si può iterare la funzione migliaia di volte. Se concateni 256 esecuzioni e salvi solo l'ultimo output, un attaccante avrà bisogno di un tempo circa 256 volte maggiore.

